Generalización de Código de Automatización en Desarrollo de Software con IA

Tema 5. Generación automática de tests y verificación de software

Índice

Esquema

Ideas clave

5.1. Introducción y objetivos

5.2. Importancia de las pruebas automáticas en desarrollo

5.3. Técnicas de generación automática de tests

5.4. Integración de pruebas generadas con flujo de desarrollo

5.5. Evaluación de la cobertura de pruebas automáticas

5.6. Casos prácticos de generación automática de tests

5.7. Referencias bibliográficas

A fondo

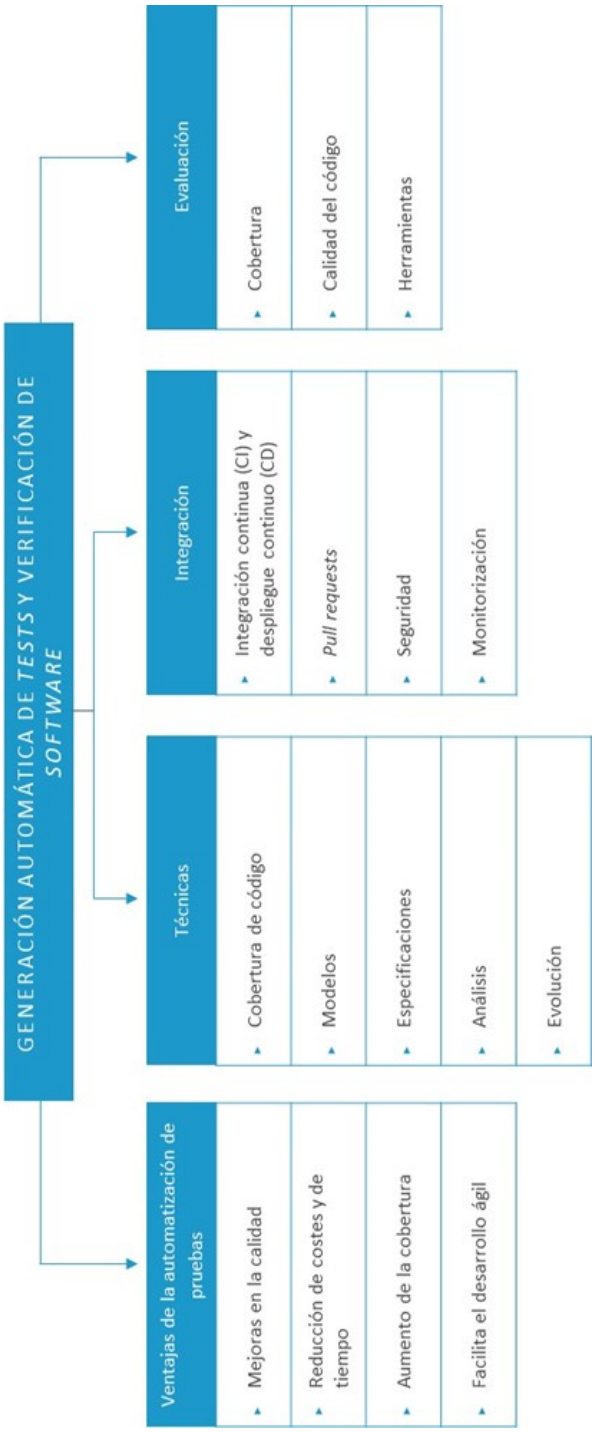
Test de seguridad en aplicaciones con Snyk

Estrategias avanzadas para pruebas automatizadas

Aplicaciones de Harness en la automatización para el diseño e implementación de test

Test

Esquema



© Universidad Internacional de La Rioja (UNIR)

5.1. Introducción y objetivos

Las **pruebas automáticas** son un componente esencial en el ciclo de vida del desarrollo de *software* moderno. Con el uso de herramientas basadas en inteligencia artificial (IA), estas pruebas **evolucionaron significativamente,** ya que mejoraron su eficiencia, cobertura y capacidad para **detectar errores en etapas tempranas** del desarrollo.

De esta manera, este tema aborda la **importancia y uso de las pruebas automáticas,** cómo la IA potencia su implementación y los **beneficios** que aportan al desarrollo de *software.*

Los objetivos de este tema son:

- Comprender la importancia de las pruebas automáticas en el desarrollo de *software.*
- Explorar las técnicas más utilizadas para la generación automática de *tests.*
- Integrar las pruebas generadas con los flujos de desarrollo existentes mediante metodologías de integración continua (CI) y de despliegue continuo (CD).
- Evaluar la cobertura de las pruebas automáticas y aprender a mejorarla a través de herramientas que generen informes detallados y detecten áreas críticas no cubiertas.
- Aplicar casos prácticos de generación automática de *tests* en distintos contextos.

© Universidad Internacional de La Rioja (UNIR)

5.2. Importancia de las pruebas automáticas en desarrollo

Las pruebas automáticas son *scripts* o programas diseñados para validar automáticamente el comportamiento del *software.* Estas pruebas verifican que los componentes, módulos o sistemas completos cumplan con los requisitos funcionales y no funcionales establecidos.

Mejora de la calidad del software

Las pruebas automáticas permiten **detectar errores y defectos** de manera temprana y continua durante el desarrollo. Al implementar estas pruebas de forma consistente, los desarrolladores pueden asegurar que cada nueva modificación en el código **no introduzca regresiones o nuevos problemas.** De esta manera, se mantiene un **estándar de calidad alto.**

Por ejemplo, en un proyecto de desarrollo ágil, se puede configurar un *pipeline* de CI que ejecute automáticamente un conjunto de pruebas unitarias y de integración cada vez que se realiza un cambio en el código. Esto garantiza que cualquier defecto que pueda surgir sea identificado antes de que se despliegue la nueva versión.

Reducción de costos a largo plazo

Detectar y corregir errores en etapas tempranas es mucho más económico que hacerlo una vez que el *software* fue desplegado en producción. Las pruebas automáticas permiten realizar estas **validaciones de forma más rápida y eficiente** que las pruebas manuales, ya que **reducen los costos** de mantenimiento y evitan problemas que puedan afectar la operación del *software.*

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Según un estudio de la consultora IBM, el costo de corregir un defecto encontrado en la fase de diseño puede ser hasta cien veces menor que si se encuentra en producción.

Eficiencia y velocidad en el desarrollo

La automatización de pruebas permite ejecutar **pruebas repetitivas y exhaustivas** de manera mucho más rápida que manualmente. Esto libera tiempo para los desarrolladores y *testers,* por lo que les permite centrarse en tareas más complejas y creativas. Además, facilita el **desarrollo rápido de nuevas funcionalidades** al integrar la **verificación constante** de que el código existente sigue funcionando correctamente.

Por ejemplo, en el desarrollo de un sistema de comercio electrónico, las pruebas automáticas pueden verificar en cada actualización que las funciones de pago, de búsqueda y de carrito de compras sigan operativas, sin necesidad de pruebas manuales repetitivas cada vez que se agregan características o se hacen correcciones.

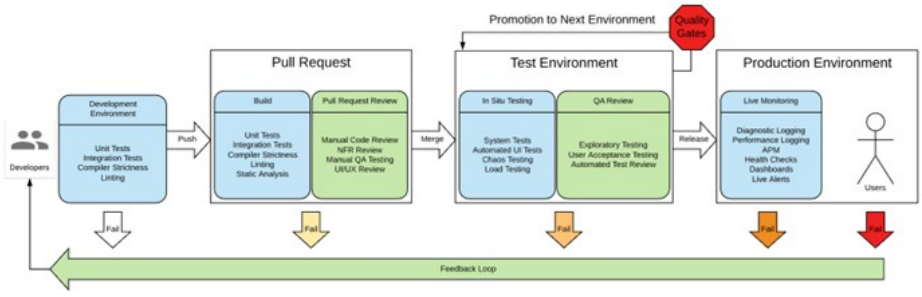


Figura 1. *Pipeline* de CI y CD con pruebas integradas. Fuente: Dancer's Code, 2019.

La Figura 1 muestra un diagrama de cómo un *pipeline* de CI y CD ejecuta pruebas automáticas después de cada commit o integración de código.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Aumento de la cobertura de pruebas

Las pruebas manuales suelen estar **limitadas** por el tiempo y los recursos disponibles, lo que significa que algunas funcionalidades del *software* pueden no probarse lo suficiente. Las pruebas automáticas permiten **aumentar significativamente la cobertura de código** para asegurar que se evalúen más casos y se verifiquen más escenarios, lo que **ayuda a detectar errores** que, de otro modo, podrían pasar desapercibidos.

Por ejemplo, en un sistema de gestión financiera, las pruebas automáticas pueden generar cientos de pruebas diferentes que abarquen diversos escenarios de cálculo de impuestos, de pagos y de procesamiento de transacciones, para asegurar que todos los casos posibles sean cubiertos.

Facilita el desarrollo ágil y DevOps

Las metodologías ágiles y los enfoques de DevOps requieren **ciclos de desarrollo rápidos y frecuentes,** lo que demanda pruebas continuas para asegurar que las nuevas versiones no introduzcan errores. Las pruebas automáticas son esenciales para sostener este **ritmo de entrega rápida** sin comprometer la estabilidad del *software.*

En los equipos que adoptan DevOps, es común que las pruebas automáticas estén integradas en el flujo de trabajo, de modo tal que cada nuevo código que se añade pase por pruebas rigurosas antes de ser desplegado automáticamente en producción.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Mejora la mantenibilidad del código

El uso de pruebas automáticas, especialmente pruebas unitarias, obliga a los desarrolladores a escribir **código más modular y desacoplado,** ya que estos son más fáciles de probar. Esto no solo mejora la **mantenibilidad del código a largo plazo,** sino que también facilita la **introducción de nuevas funcionalidades y la refactorización del código existente** sin romper funcionalidades previas.

Por ejemplo, en un proyecto de desarrollo de una API *(application programming interface)* REST *(representational state transfer)*, la creación de pruebas unitarias para cada *endpoint* permite realizar cambios en la estructura interna de la API sin temor a introducir errores, ya que las pruebas verificarán automáticamente si las respuestas siguen siendo correctas.

Mejora en la seguridad del software

La automatización de pruebas no se limita solo a las **pruebas funcionales,** sino que también incluye **pruebas de seguridad.** De esta manera, se pueden automatizar pruebas para verificar la **robustez del *software*** contra vulnerabilidades comunes, como inyecciones de código, autenticación insegura o violaciones de permisos.

Por ejemplo, en un sistema de autenticación, las pruebas automáticas pueden incluir validaciones de seguridad, como pruebas de fuerza bruta en contraseñas o pruebas de acceso indebido a recursos restringidos.

Escalabilidad del proceso de pruebas

Conforme el *software* **crece en complejidad y tamaño,** las pruebas manuales se vuelven impracticables. Las pruebas automáticas permiten **escalar el proceso de prueba** para abarcar proyectos más grandes, sin requerir un aumento proporcional en los recursos humanos.

© Universidad Internacional de La Rioja (UNIR)

Por ejemplo, en un sistema distribuido con múltiples servicios conectados, las pruebas automáticas pueden simular miles de peticiones y respuestas entre servicios para verificar el comportamiento en escenarios reales de carga.

Pruebas automatizadas en diferentes entornos

Las pruebas automáticas pueden ejecutarse en múltiples entornos y configuraciones (por ejemplo, diferentes sistemas operativos, navegadores o dispositivos), por lo que aseguran que el *software* funcione correctamente en todos los contextos en los que será utilizado.

Por ejemplo, en una aplicación web, las pruebas automáticas pueden ejecutarse en varios navegadores (Chrome, Firefox y Safari) y dispositivos (móvil, tableta y escritorio) para asegurar que la funcionalidad sea consistente en todas las plataformas.

© Universidad Internacional de La Rioja (UNIR)

5.3. Técnicas de generación automática de tests

La generación automática de *tests* es una práctica crucial para asegurar la calidad del *software.* Las herramientas tradicionales ofrecen capacidades básicas para **automatizar pruebas unitarias, funcionales e integración,** pero presentan **limitaciones** en términos de cobertura y flexibilidad.

Con la incorporación de la IA, estas técnicas evolucionaron para superar esas barreras, lo que permite la creación de pruebas más rápidas, precisas y adaptativas.

La generación automática de *tests* implica el **uso de herramientas** para crear casos de prueba que **evalúan el comportamiento del *software,*** además de que verifican que cumpla con los requisitos establecidos:

- Hay que asegurar que el código nuevo o modificado no introduzca errores.
- Validar que los sistemas cumplan con las especificaciones funcionales y de rendimiento.
- Mejorar la cobertura de las pruebas y la confiabilidad del *software.*

© Universidad Internacional de La Rioja (UNIR)

Técnicas tradicionales frente a técnicas potenciadas por IA

Técnicas tradicionales

- **Pruebas basadas en registros manuales.** Los desarrolladores o *testers* diseñan casos de prueba manualmente, por lo que definen entradas específicas y evalúan las salidas esperadas. Sus limitaciones son:

  - Consumen tiempo.
  - Dependencia del conocimiento del *tester.*
  - Cobertura limitada debido al enfoque manual.

- **Generación de *scripts* estáticos.** Hay herramientas tradicionales (como Selenium o JUnit) que permiten automatizar casos de prueba específicos definidos por los desarrolladores.

Generación de *scripts* estáticos			
Ventajas		**Desventajas**	
1	Eliminan tareas repetitivas	1	No adaptativas a cambios en el código
2	Reutilizables en diferentes ciclos de desarrollo	2	Requieren actualizaciones manuales constantes

Tabla 1. Generación de scripts estáticos. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

- **Pruebas basadas en modelos.** Los casos de prueba se derivan de modelos de comportamiento del sistema, como diagramas de estados o flujos de procesos. Limitaciones:

  - Complejidad en la creación de modelos.
  - Dificultad para manejar sistemas grandes o dinámicos.

Generación automática de *tests* potenciada por IA

La incorporación de IA transformó la generación de pruebas al permitir la creación de casos de prueba basados en **datos, patrones y comportamientos aprendidos.** Las herramientas de IA van más allá de las técnicas tradicionales al **analizar código, predecir posibles fallos y generar pruebas adaptativas.**

- **Análisis estático automatizado.** Hay herramientas, como DeepCode, que analizan el código fuente para identificar puntos críticos y generar pruebas para áreas propensas a errores. Ventajas:

  - Cobertura inmediata y exhaustiva.
  - Identificación de vulnerabilidades de seguridad, como inyecciones SQL *(structured query language)* o accesos no controlados.

- **Pruebas basadas en aprendizaje automático.** CodeT5 y GraphCodeBERT utilizan modelos de lenguaje para entender el contexto del código y generar pruebas unitarias, funcionales o de integración automáticamente.

  - Por ejemplo, se pueden generar pruebas para validar que una API REST devuelva las respuestas correctas para varios escenarios de entrada.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Pruebas basadas en datos históricos.** Hay herramientas, como Datadog y Harness, que analizan datos de producción para crear casos de prueba basados en patrones reales de uso. Las ventajas son:
  - Identificación de casos límite no previstos manualmente.
  - Mejora la robustez del *software* al reflejar escenarios reales.
- **Generación de pruebas predictivas.** Hay modelos, como GPT-4-Turbo, que predicen las áreas donde es más probable que ocurran los errores y que sugieren pruebas específicas para esas áreas.
  - Por ejemplo, se pueden crear pruebas para verificar la seguridad de una función que maneje datos sensibles en un sistema bancario.

© Universidad Internacional de La Rioja (UNIR)

Comparativa: técnicas tradicionales vs. técnicas con IA

Aspecto	Técnicas tradicionales	Técnicas con IA
Cobertura	Limitada a los casos definidos manualmente.	Amplia, ya que abarca casos complejos y basados en datos históricos.
Eficiencia	Consumen mucho tiempo y esfuerzo humano.	Rápidas, dado que tienen una generación automatizada en tiempo real.
Adaptación a cambios	Requieren actualizaciones manuales constantes.	Adaptativas, puesto que ajustan las pruebas automáticamente.
Complejidad de casos	Limitada a escenarios previstos.	Capaces de manejar casos límite y escenarios inesperados.
Identificación de errores	Basada en conocimiento del *tester* o desarrollador.	Detectan vulnerabilidades y errores complejos mediante IA.
Velocidad	Dependiente del tiempo humano para definir y ejecutar pruebas.	Automatización completa que reduce significativamente el tiempo.

Tabla 2. Técnicas tradicionales vs. técnicas con IA. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

5.4. Integración de pruebas generadas con flujo de desarrollo

La integración de **pruebas generadas automáticamente mediante IA** en el flujo de desarrollo de *software* es una práctica cada vez más común que mejora la **eficiencia, calidad y confiabilidad** de los productos. Estas pruebas, creadas con herramientas avanzadas, se integran en diversas etapas del ciclo de desarrollo para **detectar errores tempranamente,** garantizar la cobertura de pruebas y acelerar el tiempo de entrega.

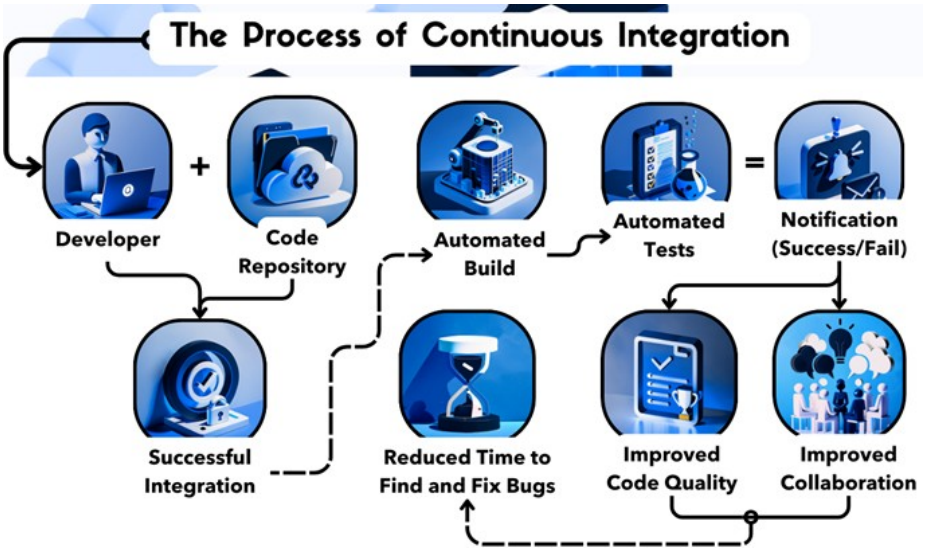


Figura 2. Gestión de *test* automatizados en un *pipeline* de CI. Fuente: Terekhov, 2024.

Las pruebas generadas por IA son casos de prueba creados automáticamente por herramientas que analizan código, datos históricos o patrones de uso para identificar escenarios relevantes y posibles errores.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Las características clave son:

- Se basan en el análisis de patrones y en el aprendizaje de datos del código.
- Pueden abarcar pruebas unitarias, de integración, funcionales y de carga.
- Se actualizan automáticamente a medida que cambia el código.

Beneficios de la integración

- Detección temprana de errores. Las pruebas generadas por IA identifican problemas en las primeras etapas del desarrollo, lo que reduce los costos y el esfuerzo necesarios para solucionarlos.
- Ahorro de tiempo. La automatización elimina la necesidad de crear manualmente casos de prueba, por lo que permiten que los desarrolladores se concentren en otras tareas críticas.
- Cobertura de pruebas más amplia. Las herramientas de IA generan pruebas que incluyen casos límite y escenarios complejos que podrían pasar desapercibidos manualmente.
- Adaptación continua. Las pruebas se actualizan automáticamente con base en los cambios realizados en el código, lo que asegura su relevancia en todo momento.
- Mayor calidad del producto. Al identificar más errores y garantizar que todas las partes del *software* sean probadas, se mejora la calidad general del producto.

© Universidad Internacional de La Rioja (UNIR)

Proceso de integración

- Análisis del código fuente. Hay herramientas, como DeepCode o SonarQube, que analizan el código para identificar áreas críticas y generar pruebas relevantes.
  - Por ejemplo, DeepCode detecta vulnerabilidades de seguridad y genera pruebas para evaluar los escenarios de ataque más probables.
- Generación de pruebas automáticas. Hay herramientas, como GitHub Copilot o CodeT5, que crean automáticamente pruebas unitarias y de integración.
  - Por ejemplo, Copilot genera una prueba unitaria en Pytest para verificar que una función de cálculo matemático maneje correctamente entradas extremas.
- Integración con CI y CD. Las pruebas generadas se integran en *pipelines* de CI y CD mediante herramientas como GitHub Actions, Jenkins o Harness.
  - Por ejemplo, GitHub Actions ejecuta pruebas automáticas en cada commit, lo que garantiza que los cambios no introduzcan errores.

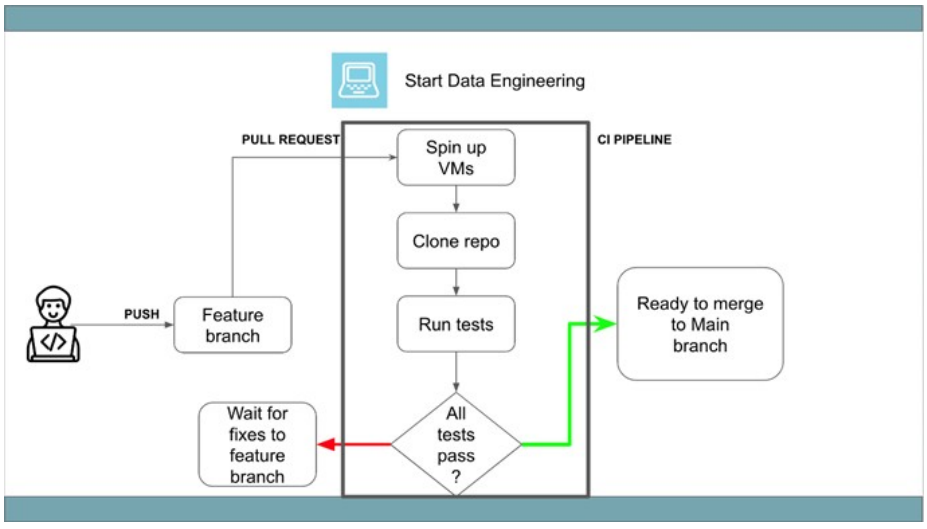


Figura 3. Gestión de pull request con *test* automatizados y gestión de errores. Fuente: Start Data Engineering, 2022.

© Universidad Internacional de La Rioja (UNIR)

- Ejecución y validación automática. Las pruebas se ejecutan automáticamente en cada etapa del flujo de desarrollo, desde el desarrollo local hasta el entorno de producción.
  - Por ejemplo, Harness valida el rendimiento del *software* bajo condiciones de carga simulada, por lo que se basa en datos reales de producción.
- Reporte y retroalimentación. Hay herramientas, como Datadog o Azure DevOps, que generan reportes detallados que ayudan a los desarrolladores a identificar y a solucionar errores rápidamente.
  - Por ejemplo, Datadog alerta sobre un aumento en los tiempos de respuesta en una API y proporciona un historial de pruebas para rastrear la causa.

Ejemplo práctico: implementación de pruebas generadas por IA en CI y CD

Contexto

Un equipo de desarrollo trabaja en una aplicación de comercio electrónico que recibe constantes actualizaciones y necesita mantener la calidad del *software* bajo alta presión de entrega.

Solución

- Integración de herramientas:
  - GitHub Copilot genera automáticamente pruebas unitarias para cada nueva función añadida.
  - Harness simula cargas de usuarios basadas en datos históricos para garantizar el rendimiento.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- *Pipeline* de CI y CD:
  - Se utiliza GitHub Actions para ejecutar automáticamente las pruebas generadas por Copilot y Harness en cada commit .
  - SonarQube analiza la calidad del código antes del despliegue.
- Ejecución automática. Cada vez que se actualiza el código, el *pipeline* ejecuta las pruebas y genera los reportes automáticos.
- Resultados:
  - Reducción del 40 % del tiempo necesario para validar cambios.
  - Identificación temprana de errores en módulos críticos, lo que evita problemas en producción.

© Universidad Internacional de La Rioja (UNIR)

5.5. Evaluación de la cobertura de pruebas automáticas

La cobertura de pruebas es una métrica crucial en el desarrollo de *software,* utilizada para medir **qué tan exhaustivamente se evaluó el código** con los casos de prueba existentes. Con la incorporación de herramientas de IA en la generación de pruebas, la evaluación de cobertura evolucionó, lo que permitió **análisis más detallados y dinámicos** que ayudan a mejorar la calidad del *software.* De esta manera, abordaremos la importancia de evaluar la cobertura, los métodos utilizados y las **ventajas de aplicar herramientas basadas en IA.**

La cobertura de pruebas mide la proporción del código fuente que fue ejecutado durante la ejecución de los casos de prueba. Su objetivo principal es identificar las áreas del *software* que no fueron probadas, para minimizar los riesgos de errores en producción.

Tipos de cobertura de pruebas

Existen varios tipos de cobertura de pruebas, cada uno enfocado en diferentes aspectos del código:

- La cobertura de líneas mide el porcentaje de líneas de código que fueron ejecutadas durante las pruebas. Esta es la métrica más básica y común.
  - Por ejemplo, si un archivo de código tiene cien líneas y las pruebas ejecutan 75 de estas, la cobertura de líneas será del 75 %.
- La cobertura de sentencias es similar a la cobertura de líneas, pero, en este caso, mide las sentencias de código ejecutadas. Algunas líneas pueden tener varias sentencias, lo que hace que esta métrica sea más específica.

© Universidad Internacional de La Rioja (UNIR)

- La cobertura de ramas mide si todas las ramas de las estructuras de control condicionales (como `if`, `else` y `switch`) fueron ejecutadas durante las pruebas.
  - Por ejemplo, si una condición `if else` tiene dos posibles caminos y las pruebas solo cubren uno, la cobertura de ramas sería del 50 %.
- La cobertura de funciones evalúa si todas las funciones del código fueron llamadas durante las pruebas.
- La cobertura de condiciones verifica si todas las condiciones posibles dentro de expresiones lógicas fueron probadas.
  - Por ejemplo, en una expresión condicional con múltiples términos (por ejemplo, `if (a && b)`), la cobertura de condiciones se asegura de que ambas, `a` y `b`, sean probadas en ambas posibilidades (verdadero y falso).

Herramientas para medir cobertura

Las **herramientas basadas en IA** transformaron el proceso de medición de la cobertura de pruebas al ofrecer análisis más precisos y dinámicos. Estas herramientas **identifican áreas no probadas,** generan métricas detalladas y **optimizan la cobertura,** por lo que ayudan a los equipos a mantener altos estándares de calidad.

SonarQube

- Descripción. Herramienta de análisis de calidad de código que evalúa la cobertura de líneas, las ramas y los métodos mediante reportes visuales y detallados.
- Características clave:
  - Identificación automática de áreas no probadas.
  - Generación de métricas de cobertura con recomendaciones para mejorar.
  - Integración con *pipelines* de CI y CD.

© Universidad Internacional de La Rioja (UNIR)

- Casos de uso. Detectar funciones críticas de una API que no están cubiertas por pruebas.
- Ventajas:
  - Interfaz visual intuitiva.
  - Compatible con múltiples lenguajes de programación.

Codecov

- Descripción. Plataforma de cobertura de código que se integra con repositorios como GitHub, GitLab y Bitbucket.
- Características clave:
  - Reportes visuales de cobertura en tiempo real.
  - Análisis de áreas no probadas en nuevas funciones o cambios recientes.
  - Compatible con herramientas de CI y CD.
- Casos de uso:
  - Evaluar la cobertura de ramas y de líneas en un proyecto de desarrollo continuo.
- Ventajas:
  - Fácil integración con repositorios.
  - Compatible con herramientas de pruebas como Jest y Pytest.

© Universidad Internacional de La Rioja (UNIR)

Harness

- Descripción. Herramienta de IA que utiliza datos reales de producción para evaluar la cobertura y generar pruebas basadas en patrones de uso.
- Características clave:
  - Simulación de escenarios reales en entornos controlados.
  - Identificación de áreas críticas no probadas.
  - Pruebas de rendimiento y carga basadas en datos históricos.
- Casos de uso. Validar la cobertura de funciones bajo condiciones de carga simulada.
- Ventajas:
  - Foco en escenarios reales.
  - Ideal para aplicaciones de alta concurrencia.

DeepCode (Snyk)

- Descripción. Herramienta que analiza el código fuente a través de IA para detectar vulnerabilidades y evaluar la cobertura en las áreas críticas.
- Características clave:
  - Generación de pruebas específicas para rutas no probadas.
  - Identificación de errores de seguridad y recomendaciones automáticas.
- Casos de uso. Hay que asegurar que todas las rutas de un sistema de autenticación estén probadas.

© Universidad Internacional de La Rioja (UNIR)

- Ventajas:
  - Análisis profundo de vulnerabilidades.
  - Integración con IDE (entorno de desarrollo integrado) y pipelines de CI y CD.

Datadog

- Descripción. Plataforma de monitorización y evaluación que utiliza IA para analizar patrones de uso y evaluar la cobertura de pruebas.
- Características clave:
  - Identificación de áreas del código más utilizadas en producción.
  - Priorización de pruebas en funciones críticas.
  - Reportes en tiempo real sobre cobertura y rendimiento.
- Casos de uso. Validar que las rutas más frecuentadas por los usuarios están completamente probadas.
- Ventajas:
  - Análisis basado en datos reales.
  - Integración con herramientas de despliegue continuo.

© Universidad Internacional de La Rioja (UNIR)

Azure DevOps con IA

- Descripción. Plataforma completa de desarrollo que utiliza IA para analizar métricas de calidad y cobertura en proyectos integrados.
- Características clave:
  - Reportes de cobertura de pruebas basados en cambios recientes.
  - Priorización de pruebas según el impacto esperado.
- Casos de uso. Generar métricas de cobertura antes de un despliegue importante.
- Ventajas:
  - Integración con herramientas de CI y CD.
  - Enfoque en la colaboración entre equipos.

GraphCodeBERT

- Descripción. Modelo de IA diseñado para comprender y analizar código a través de grafos de sintaxis abstracta (AST).
- Características clave:
  - Identificación de rutas no probadas en estructuras complejas de código.
  - Generación de pruebas basadas en patrones aprendidos del código.
- Casos de Uso:
  - Mejorar la cobertura de rutas condicionales en sistemas complejos.

© Universidad Internacional de La Rioja (UNIR)

- Ventajas:
  - Enfoque en estructuras de código complejas.
  - Compatible con múltiples lenguajes.

Comparativa

Herramienta	Métrica principal	Diferenciador	Ventajas
SonarQube	Líneas, ramas y métodos.	Análisis visual y recomendaciones.	Interfaz intuitiva y multiidioma.
Codecov	Líneas y ramas en tiempo real.	Integración con repositorios y CI y CD.	Visualización en tiempo real.
Harness	Cobertura en escenarios reales.	Simulación basada en datos históricos.	Foco en cargas reales.
DeepCode (Snyk)	Vulnerabilidades y rutas críticas.	Enfoque en seguridad y calidad del código.	Integración con IDE y CI y CD.
Datadog	Áreas más utilizadas en producción.	Priorización basada en datos reales.	Ideal para entornos en producción.
Azure DevOps	Cobertura por impacto de cambios.	Reportes de colaboración entre equipos.	Integración con flujos completos.
GraphCode BERT	Cobertura en rutas condicionales.	Análisis profundo con grafos AST.	Ideal para código complejo.

Tabla 3. Comparativa. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Las herramientas basadas en IA, como SonarQube, GitHub Copilot y Harness, revolucionaron la **evaluación de la cobertura de pruebas** al automatizar análisis, detectar áreas críticas y priorizar casos de prueba en función de datos reales. Su integración en los flujos de desarrollo mejora la calidad del *software,* reduce errores y **optimiza el tiempo** de los equipos de desarrollo. Al adoptar estas herramientas, las organizaciones pueden alcanzar niveles superiores de **confiabilidad y eficiencia** en sus proyectos.

© Universidad Internacional de La Rioja (UNIR)

5.6. Casos prácticos de generación automática de tests

La generación automática de *tests* es una técnica avanzada que les permite a los desarrolladores **crear casos de prueba** de manera automatizada, en lugar de hacerlo manualmente. Este enfoque mejora la **eficiencia,** la **cobertura** y la **calidad del código,** mientras que reduce errores humanos. A continuación, se presentan algunos casos prácticos de cómo la generación automática de *tests* puede aplicarse en diferentes escenarios de desarrollo de *software:*

Generación de pruebas unitarias con GitHub Copilot

Supongamos que tenemos una función de Python para comprobar que el precio de algo es mayor que cero o menor o igual a mil. Usemos Copilot para generar casos de prueba para ello (Larkin, 2024).

```
def validate_price(price: float):

if price <= 0:

   raise ValueError("Price must be greater than 0")

if price > 1000:

   raise ValueError("Price must be less than or equal to 1000")
```

Con Copilot habilitado, deberá ingresar el siguiente comando y mensaje en Copilot Chat: /tests create tests for validate_price . Verifique también los casos extremos.

Desde allí, verá un resultado como este:

```
import unittest

from generate_unit_tests import validate_price
```


© Universidad Internacional de La Rioja (UNIR)

```
class TestValidatePrice(unittest.TestCase):

def test_price_zero(self):

   with self.assertRaises(ValueError) as context:

     validate_price(0)

   self.assertEqual(str(context.exception), "Price must be greater than 0")

def test_price_negative(self):

   with self.assertRaises(ValueError) as context:

     validate_price(-10)

   self.assertEqual(str(context.exception), "Price must be greater than 0")

def test_price_above_limit(self):

   with self.assertRaises(ValueError) as context:

     validate_price(1500)

   self.assertEqual(str(context.exception), "Price must be less than or
equal to 1000")

def test_price_edge_case_zero(self):

   with self.assertRaises(ValueError) as context:

     validate_price(0)

   self.assertEqual(str(context.exception), "Price must be greater than 0")

def test_price_edge_case_max(self):

   try:

     validate_price(1000)
```


© Universidad Internacional de La Rioja (UNIR)

```
    except ValueError:
        self.fail("validate_price() raised ValueError unexpectedly!")
if __name__ == '__main__':
unittest.main()
```

Explicación de las pruebas generadas

Aquí vemos que GitHub Copilot genera pruebas unitarias para validar el comportamiento de la función validation_price . A continuación, se muestra un desglose de lo que hace cada prueba:

- test_price_zero confirma que, al llamar a validation_price(0), se genera un ValueError con el mensaje «El precio debe ser mayor que ». Esto prueba la condición de límite inferior para un precio no válido.
- test_price_negative confirma que, al llamar a validation_price(-10) , se genera un ValueError con el mismo mensaje, lo que garantiza que los valores negativos se manejen correctamente.
- test_price_above_limit valida que, al llamar a validation_price(1500) , se genere un ValueError con el mensaje «El precio debe ser menor o igual a 1000 », probando el límite superior en busca de precios no válidos.
- test_price_edge_case_zero es un duplicado de test_price_zero (verifica la misma condición). Es redundante y se puede eliminar.
- test_price_edge_case_max verifica que la llamada a validation_price(1000) no genere un error. Si se genera una excepción, la prueba falla.

Si bien estas pruebas funcionan, también se pueden mejorar con un mensaje de seguimiento para eliminar redundancias (como test_price_edge_case_zeroy test_price_zero) y agregar una prueba para validar los precios dentro del rango

© Universidad Internacional de La Rioja (UNIR)

aceptable (por ejemplo, `validate_price(500)`). Recuerde también que GitHub Copilot funciona con LLM (modelos extensos de lenguaje) no deterministas que no siempre producen los mismos resultados, y siempre se debe revisar el código sugerido por Copilot, así como volver a solicitarle a Copilot que mejore ese código.

Cobertura de código con Codecov

En este tutorial, aprenderás a configurar la cobertura de código en un proyecto de Unity mediante el paquete Code Coverage y cómo integrar Codecov con su repositorio de GitHub a través de GitHub Actions para generar informes de cobertura cada vez que envía cambios o realiza una solicitud de extracción (Chagialas, 2021).

Tareas:

- Configuración del proyecto.
- Ejecución de pruebas y generación de un informe de cobertura localmente.
- Configuración de GitHub Actions.
- Configuración de Codecov.

Configuración del proyecto

- Crea un nuevo proyecto de Unity. Inicie Unity Hub y abra un proyecto existente o crea un nuevo proyecto 3D vacío de Unity. Code Coverage está disponible en la versión 2019.3 de Unity y posteriores.
- Instale el paquete cobertura del código. Utilice el «Administrador de paquetes de Unity» («Ventana» > «Administrador de paquetes») para buscar e instalar el paquete de cobertura de código.

© Universidad Internacional de La Rioja (UNIR)

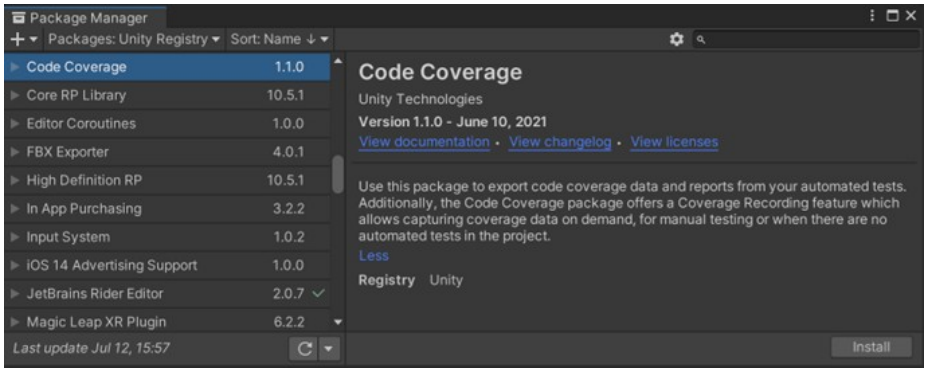


Figura 4. Code Coverage. Fuente: elaboración propia.

Como alternativa, utilice el menú desplegable «Agregar(+)» y seleccione «Agregar paquete desde URL de git…» o «Agregar paquete por nombre…» y escriba com.unity.testtools.codecoverage .

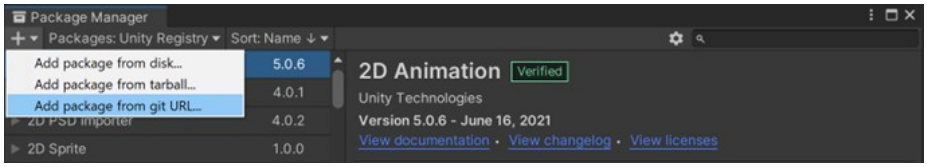


Figura 5. Code Coverage. Fuente: elaboración propia.

- Instalar el proyecto de ejemplo Asteroids. En el «Administrador de paquetes de Unity» («Ventana» > «Administrador de paquetes»), seleccione el paquete «Cobertura de código», si aún no está seleccionado. Busca la sección «Samples» en los detalles del paquete (lado derecho) y seleccione «Importar junto a tutorial de cobertura de código».

© Universidad Internacional de La Rioja (UNIR)

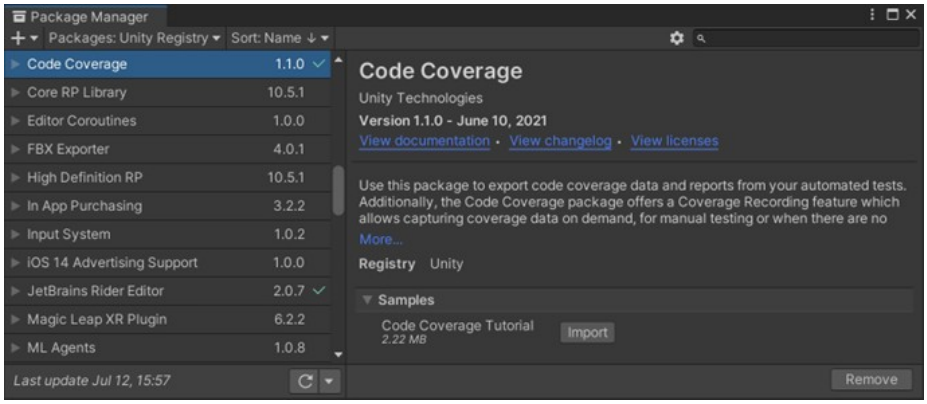


Figura 6. Code Coverage. Fuente: elaboración propia.

Ejecución de pruebas y generación de un informe de cobertura localmente

- Habilitar cobertura de código. Para habilitar la cobertura de código, abra la ventana «Cobertura de código» («Ventana» > «Análisis» > «Cobertura de código») y seleccione «Habilitar cobertura de código», si aún no está seleccionado, para poder generar datos e informes de cobertura.
  - Si ve esta advertencia, seleccione «Cambiar al modo de depuración».
- Generar un informe de cobertura a partir de las pruebas de «PlayMode»:
  - Asegúrese de que solo los paquetes `Unity.TestTools.CodeCoverage.Sample.Asteroids` y `Unity.TestTools.CodeCoverage.Sample.Asteroids.Tests` estén seleccionados en «Included Assemblies».
  - Asegúrese de que las opciones «Generar informe HTML» y «Generar informe automáticamente» estén marcadas.

© Universidad Internacional de La Rioja (UNIR)

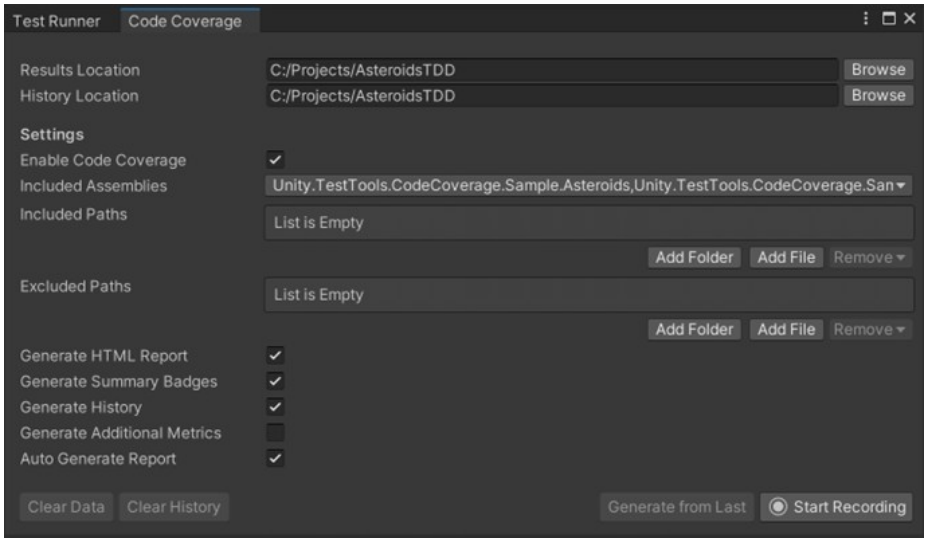


Figura 7. Code Coverage. Fuente: elaboración propia.

- Cambia a la ventana «Test runner», seleccione la pestaña «PlayMode» y presione «Ejecutar todas las pruebas».
- Cuando las pruebas terminen de ejecutarse, se abrirá una ventana de visualización de archivos que contiene el informe de cobertura. Seleccione `index.htm` .
- Busca las clases con baja cobertura, especialmente `LaserController` , `BaseProjectile` y `ProjectileController` . Puede ordenar los resultados por «Cobertura de línea» y hacer clic en cada clase para obtener un informe más detallado.

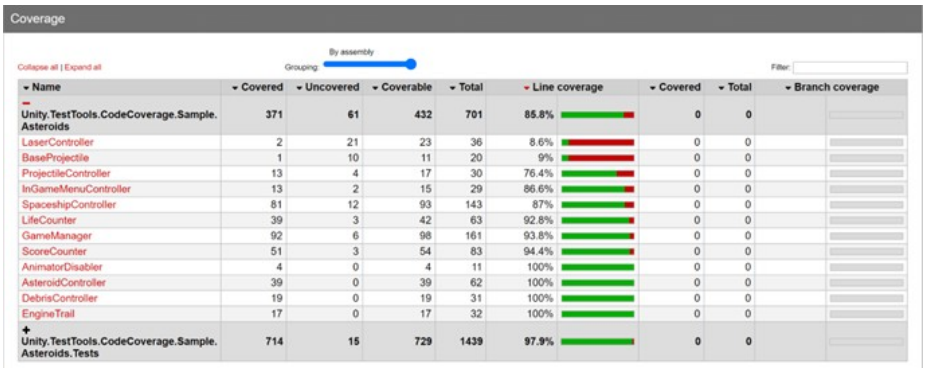


Figura 8. Code Coverage. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Configuración de GitHub Actions

- Crea un repositorio en GitHub y envía el proyecto allí. Se recomienda incluir la plantilla de Unity gitignore de GitHub. También puedes agregar en gitignore: /[Cc]ode[Cc]overage/ .

- Configurar GitHub Actions. No existe una forma oficialmente compatible de ejecutar Unity en GitHub Actions, pero la comunidad creó algunas soluciones. Para este tutorial, usaremos GitHub Actions proporcionado por GameCI.

  - Configurar una licencia de Unity: todas las acciones usan una instalación de Unity, que debe activarse. Siga las instrucciones en la página de activación en GameCI para configurar una licencia de Unity.

  - Configurar el flujo de trabajo principal: crea un archivo llamado .github/workflows/main.yml y agregue la definición del flujo de trabajo a continuación:

```
name: CI

on:

  # Replace with your main branch if needed

  push:

    branches:

      - main

  pull_request:

    branches:

      - main

jobs:
```


© Universidad Internacional de La Rioja (UNIR)

```
runTests:
  name: Run Tests
  runs-on: ubuntu-latest
  steps:
    - name: Checkout repository
      uses: actions/checkout@v2
      with:
        lfs: true
    - name: Cache Library
      uses: actions/cache@v2
      with:
        path: Library
        key: Library
    - name: Run tests and generate Coverage XML report
      uses: game-ci/unity-test-runner@v2
      env:
        UNITY_LICENSE: ${{ secrets.UNITY_LICENSE }}
      with:
   # Replace with the Unity version your project should be tested on.
        # Minimum version: 2019.3
```


© Universidad Internacional de La Rioja (UNIR)

```
          unityVersion: 2020.3.13f1

          testMode: playmode

          customParameters: -debugCodeOptimization -enableCodeCoverage -
coverageResultsPath ./coverage-results -coverageOptions
generateAdditionalMetrics
```

Este flujo de trabajo se activa cuando se envía código a la mainrama del repositorio o se crea o actualiza una solicitud de incorporación de cambios. Contiene un runTeststrabajo que consta de tres pasos:

- Visita el repositorio.
- Almacenar en caché la carpeta de la biblioteca.
- Ejecute las pruebas «PlayMode» y genere un informe XML de cobertura.

Confirma (commit) y envía la definición de tu flujo de trabajo. Ve a la pestaña «Actions» en GitHub y selecciona el flujo de trabajo del último commit .

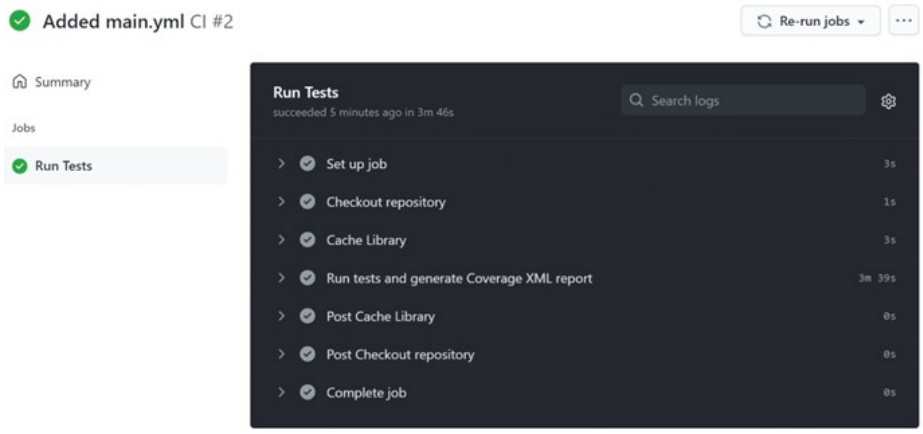


Figura 9. Code Coverage. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Configuración de Codecov

Regístrate en Codecov. Si aún no tienes una cuenta, ve a Codecov y regístrate con tu cuenta de GitHub. Una vez que lo hagas, verás una lista de tus repositorios de GitHub. Elige el repositorio que acabas de crear para vincularlo con Codecov. Se te mostrará un *token* de carga. Si tu repositorio es privado, debes copiar el *token* de carga del panel de Codecov y pegarlo en un secreto cifrado de GitHub llamado CODECOV_TOKEN .

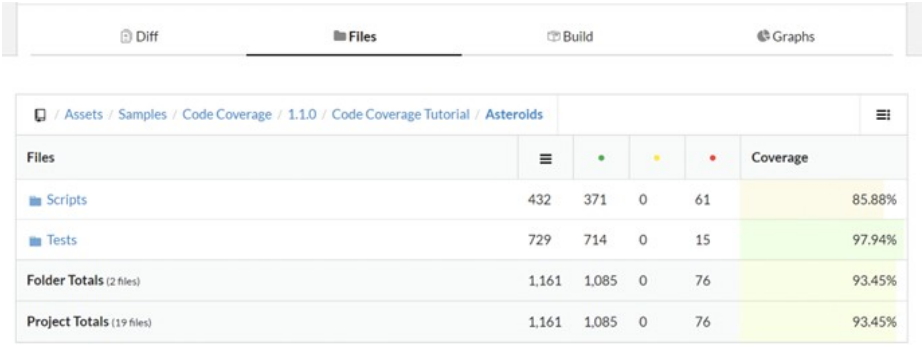
Configurar la acción de GitHub de Codecov. Ahora vamos a agregar un nuevo paso en el flujo de trabajo, que cargará el informe XML de cobertura a Codecov:

```
- name: Upload XML report to Codecov
    uses: codecov/codecov-action@v2
    with:
      name: PlayMode
      flags: automated
      token: ${{ secrets.CODECOV_TOKEN }}
      files: coverage-results/**/*.xml
```

Confirme y envíe (commit and push) la definición de flujo de trabajo actualizada.

Ver informe de Codecov. Cuando finalice el nuevo flujo de trabajo, navegue hasta el repositorio en el panel de Codecov y seleccione el último commit .

© Universidad Internacional de La Rioja (UNIR)



- Integración con pull requests :
  - Otorgue acceso al bot Codecov a su cuenta.
  - Crea una nueva rama y nómbrala add-weapon-tests .
  - Agrega Weapon tests para mejorar la cobertura; abre el Tests/WeaponTests.cs *script* y descomenta todas las pruebas (desde la línea 35 hasta la línea 237).
  - Confirma y envía los cambios.
  - Crea una pull request desde la nueva rama. Deberías ver un informe como el que se muestra a continuación:

Conclusión

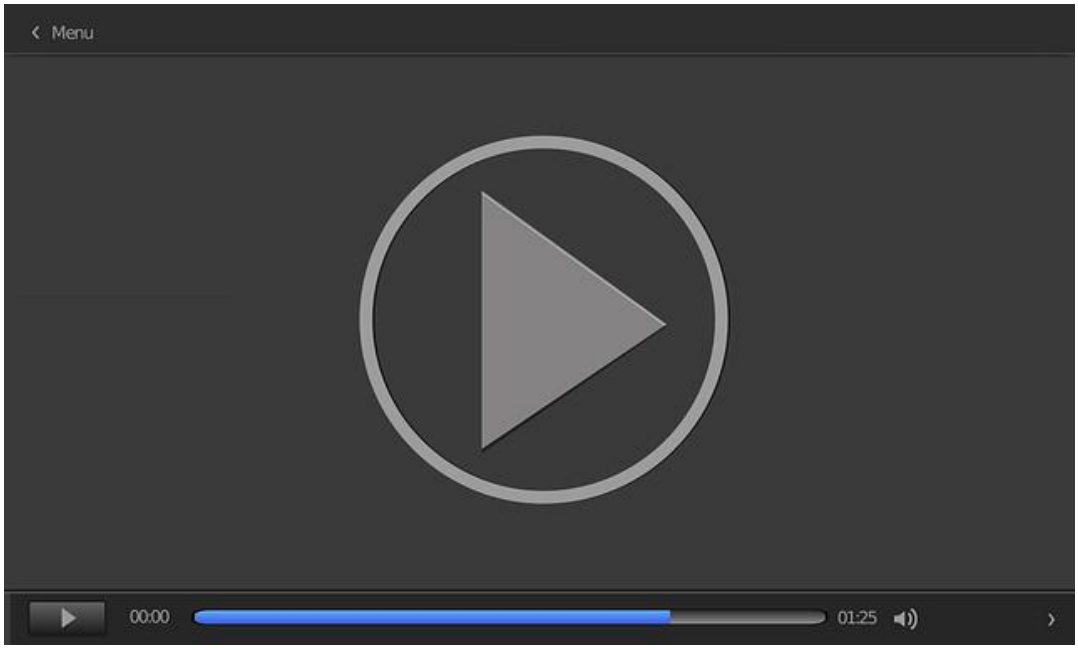
En este tutorial, aprendiste por qué la cobertura de código es importante para identificar las partes de tu código que no están cubiertas por pruebas automatizadas. La cobertura de código te permite entender dónde fallan tus pruebas y te ayuda a detectar errores de manera temprana.

También aprendimos a configurar la cobertura de código en un proyecto de Unity mediante el paquete Code Coverage y a integrar Codecov con su repositorio de GitHub a través de GitHub Actions para generar informes de cobertura cada vez que envía cambios o realiza una solicitud de extracción.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

A continuación, veremos el vídeo *Prompt Engineering para generación de pruebas unitarias.*



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=3d460f05-64a5-49bc-8916-b24201278f68

© Universidad Internacional de La Rioja (UNIR)

5.7. Referencias bibliográficas

Chagialas, N. (2021, septiembre 9). *How to Set Up Codecov with Unity and GitHub Actions.* Codecov. https://about.codecov.io/blog/how-to-set-up-codecov-with-unity-and-github-actions/

Dancer's Code. (2019, noviembre 14). *Role of Test Automation in a CI/CD Pipeline.* https://dancerscode.com/posts/role-of-test-automation-in-a-ci-cd-pipeline/

Larkin, G. (2024, diciembre 5). *How to generate unit tests with GitHub Copilot: Tips and examples.* GitHub. https://github.blog/ai-and-ml/how-to-generate-unit-tests-with-github-copilot-tips-and-examples/

Start Data Engineering. (2022, febrero 22). *Automating data testing with CI pipelines, using Github Actions.* https://www.startdataengineering.com/post/ci-data-test/

Terekhov, V. (2024, mayo 27). *Understanding the Fundamentals of Continuous Integration and Continuous Delivery.* Attract Group. https://attractgroup.com/blog/understanding-the-fundamentals-of-continuous-integration-and-continuous-delivery/

© Universidad Internacional de La Rioja (UNIR)

Test de seguridad en aplicaciones con Snyk

Tal, L. (2020, octubre 20). *Application Security Testing (AST) - Top Questions Answered.* Snyk. https://snyk.io/learn/application-security/testing/

Este artículo de la empresa Snyk ilustra las principales técnicas y métodos empleados para realizar test de seguridad sobre las aplicaciones desarrolladas.

© Universidad Internacional de La Rioja (UNIR)

Estrategias avanzadas para pruebas automatizadas

Whittle, B. (s. f.). *Advanced testing automation strategies with GitHub Actions.* https://resources.github.com/learn/pathways/automation/intermediate/advanced-testing-strategies-with-github-actions/

Bekah Whittle, directora de servicios de campo de GitHub, ofrece esta guía donde detalla las técnicas más avanzadas para pruebas automatizadas mediante sus herramientas en GitHub Actions.

© Universidad Internacional de La Rioja (UNIR)

Aplicaciones de Harness en la automatización para el diseño e implementación de test

Harness (s. f.). *Build and Test Automation.* https://www.harness.io/learn/use-cases/build-and-test-automation

Desde la página oficial de Harness, nos llega esta interesante guía donde exploran distintos casos de uso de la aplicación Harness para el diseño y aplicación de *test* automáticos en *software.*

© Universidad Internacional de La Rioja (UNIR)

Test

1. ¿Cuál es el principal objetivo de las pruebas automáticas?

   A. Reducir el costo del *hardware.*

   B. Validar automáticamente que el *software* cumple con los requisitos establecidos.

   C. Generar informes para clientes.

   D. Incrementar el tamaño del código.

2. ¿Qué técnica tradicional permite crear casos de prueba manualmente?

   A. Pruebas basadas en modelos.

   B. Generación de *scripts* estáticos.

   C. Pruebas basadas en registros manuales.

   D. Pruebas predictivas con IA.

3. ¿Cuál de las siguientes herramientas genera automáticamente pruebas predictivas?

   A. GitHub Copilot.

   B. SonarQube.

   C. Harness.

   D. GPT-4-Turbo.

4. ¿Qué mide la cobertura de ramas en las pruebas?

   A. La cantidad de sentencias ejecutadas en el código.

   B. Las rutas de ejecución en estructuras condicionales.

   C. El tiempo de ejecución de las pruebas.

   D. El porcentaje de funciones utilizadas.

© Universidad Internacional de La Rioja (UNIR)

5. ¿Qué herramienta de IA utiliza datos de producción para crear pruebas?

A. Datadog.

B. Codecov.

C. Harness.

D. DeepCode.

6. ¿Qué ventaja principal aporta la generación automática de pruebas con IA?

A. Reduce el esfuerzo humano y aumenta la cobertura.

B. Elimina la necesidad de pruebas manuales.

C. Genera reportes financieros automáticamente.

D. Mejora el diseño gráfico del código.

7. ¿Qué tipo de pruebas valida la funcionalidad de cada componente del *software*?

A. Las pruebas unitarias.

B. Las pruebas de integración.

C. Las pruebas de rendimiento.

D. Las pruebas de seguridad.

8. ¿Qué herramienta puede generar pruebas para sistemas distribuidos bajo alta concurrencia?

A. DeepCode.

B. GraphCodeBERT.

C. Harness.

D. Azure DevOps.

© Universidad Internacional de La Rioja (UNIR)

9. ¿Cuál de estas es una métrica clave en la cobertura de pruebas?

A. Cobertura de estilo de código.

B. Cobertura de funciones.

C. Cobertura de diseño.

D. Cobertura de velocidad.

10. ¿Qué permite SonarQube en el proceso de pruebas automáticas?

A. Monitorear el tráfico de usuarios en tiempo real.

B. Detectar áreas críticas no cubiertas por las pruebas.

C. Automatizar el diseño de interfaces.

D. Ejecutar pruebas de *marketing.*

11. ¿Qué técnica tradicional presenta desventajas al no adaptarse a cambios en el código?

A. Pruebas predictivas.

B. Generación de *scripts* estáticos.

C. Pruebas basadas en datos históricos.

D. Análisis estático automatizado.

12. ¿Qué herramienta genera métricas visuales de cobertura y resalta líneas no probadas?

A. Codecov.

B. Datadog.

C. Azure DevOps.

D. DeepCode.

© Universidad Internacional de La Rioja (UNIR)

13. ¿Qué herramienta es ideal para análisis de vulnerabilidades de seguridad?

A. GitHub Copilot.

B. Harness.

C. DeepCode.

D. GraphCodeBERT.

14. ¿Qué herramienta simula escenarios de carga basados en datos reales?

A. SonarQube.

B. Harness.

C. GPT-4-Turbo.

D. Codecov.

15. ¿Qué herramienta utiliza grafos de AST para analizar código?

A. Datadog.

B. Azure DevOps.

C. GraphCodeBERT.

D. Harness.

16. ¿Qué ventaja ofrece la integración de pruebas generadas con CI y CD?

A. Reduce la necesidad de monitoreo.

B. Detecta errores en etapas tempranas.

C. Genera pruebas solo en entornos locales.

D. Mejora la interfaz de usuario.

© Universidad Internacional de La Rioja (UNIR)

17. ¿Qué técnica de IA genera pruebas basándose en patrones aprendidos?

A. Pruebas predictivas.

B. Pruebas basadas en datos históricos.

C. Pruebas basadas en modelos.

D. Pruebas basadas en aprendizaje automático.

18. ¿Qué herramienta genera alertas basadas en tiempos de respuesta anormales?

A. Datadog.

B. Codecov.

C. Harness.

D. SonarQube.

19. ¿Cuál es una limitación de las pruebas basadas en modelos?

A. Dificultad para manejar sistemas pequeños.

B. Necesidad de múltiples modelos para áreas críticas.

C. Complejidad en la creación de modelos para sistemas grandes.

D. Baja eficiencia en análisis de código.

20. ¿Qué herramienta genera automáticamente pruebas unitarias a partir del código escrito?

A. DeepCode.

B. GitHub Copilot.

C. Harness.

D. SonarQube.

© Universidad Internacional de La Rioja (UNIR)